



Australian
National
University

COMP8620

Advanced Topics in AI

Assignment 1 S2 2025

Author	Yang LIN
UID	U7418662
Date	21 August 2025

PART A

1) Proof: The visibility graph of a motion plan generates the shortest plan.

Assumption:

- A point robot operating in 2D workspace.
- The workspace contains polygonal obstacles, and the free space F be the set of points not inside any obstacle.
- A visibility graph (V, E) is an undirected graph where:
 - o The **Nodes** V consist of all obstacle vertices, plus the initial point I and goal point G .
 - o An **edge** E is added between any pair of nodes $(u, v) \in \text{Edges}$ exists **iff** the straight-line segment between u and v lies entirely in F (i.e., they are *mutually visible*)
 - o Each edge is labelled with its **Euclidean length** $\|u-v\|$, representing the true cost of moving between the points.

Step 1: The shortest collision-free path consists of straight-line segments.

In polygonal environments:

- Any shortest path from I and G must be composed of straight-line segments that:
 - o Lie entirely in the free space F , and
 - o Turn only at obstacle vertices or at I and G .
- Therefore, the shortest collision-free path is a polygonal chain with vertices from V .

Step 2: The visibility graph captures all such valid paths.

- The visibility graph includes an edge for every pair of mutually visible vertices.
- Thus, any valid polygonal path from I to G , composed of visible segments, is fully represented as a path in the graph.

Step 3: Edge weights reflect true geometric cost.

- Each edge $(u, v) \in E$ is weighted by the Euclidean distance between its endpoints.
- Therefore, the cost of a path in the graph equals the actual geometric length of that path in the workspace.

Step 4: Applying shortest path search guarantees optimality.

- Let $q_I \in V$ be the nearest visible vertex to I (i.e., $\overline{Iq_I} \subseteq F$).
- Let $q_G \in V$ be the nearest visible vertex to G (i.e., $\overline{q_G G} \subseteq F$).
- Use a graph search algorithm (Dijkstra's or A*) to find the **shortest path from q_I to q_G** in the graph.
- The full path from I to G is then:
$$path = \overline{Iq_I} + \text{Shortest path from } q_I \text{ to } q_G + \overline{q_G G}$$
- Since the graph contains all feasible paths and exact distances, the returned path is **globally optimal** in terms of length.

Conclusion: The visibility graph includes all valid straight-line, collision-free connections, and edge weights correspond to true distances. Therefore, by finding the shortest path in this graph, the visibility graph method guarantees the shortest collision-free path from I to G .

2) Determine whether the method used is suitable or not and provide explanation.

a) Visibility graph for triangle-shaped robot in 2D workspace:

It is **NOT** suitable.

Reason: The visibility graph method is designed for a point robot with polygonal obstacles. A triangle-shaped robot has non-negligible geometry, meaning that even if the robot's reference point avoids obstacles, parts of its body may still collide due to its shapes and orientation. This requires planning in the C-space, where the robot is treated as a point, but the obstacles are replaced with forbidden regions.

Better Alternative: Probabilistic Roadmap Method (PRM)

A more suitable method is the PRM, which is graph-based and sampling-driven:

- **Sampling a Roadmap:** Randomly generates collision-free configurations of the triangle-shaped robot and connects nearby ones to build a roadmap graph of the free space.
- **Using a Collision Checker:** The robot's shape is directly handled by a collision checker that validates sampled configurations and edges, without explicitly constructing the full configuration space.
- **Handling Multiple Queries:** Once built, the roadmap can efficiently answer many start-goal queries with simple graph search.

Thus, PRM is **better suited** for this problem because it avoids the complexity of explicitly building the configuration space for a non-point robot, and it naturally supports multiple motion-planning queries in the same environment.

b) PRM for point robot translation with multiple queries in a 2D workspace:

It is **suitable**.

Reason:

- PRM is a **multi-query** planner. It works by constructing a roadmap of the free configuration space during a pre-computation phase, which can then be used to efficiently answer many path planning queries between different start and goal configurations.
- For a point robot that can only translate in a 2D workspace, the **configuration space** is simply the 2D workspace itself. Obstacles in the workspace correspond directly to forbidden regions in the configuration space. The PRM algorithm generates random free configurations (nodes), connects them to nearby nodes if a collision-free path exists (edges), and stores this roadmap.

- 3) **Yes**, such a free space is possible. A free space component with a narrow passage would fit Mr. T's description.

This can be explained using the formal concept of $(\epsilon, \alpha, \beta)$ -expansiveness, which characterizes how well the structure of the free space supports both coverage and connectivity.

A free space is said to be ϵ -good if, for any point q in the free space F , the volume of its visibility region $\text{Vis}(q)$ is at least $\epsilon \cdot \text{Vol}(F)$. This implies that each point can “see” a significant portion of the space, making random connections easier. However, in spaces with **narrow passages**—regions with small clearance around obstacles—this property fails because the visibility region is very limited. Although the large open regions are well sampled (i.e., the roadmap covers the space), the narrow passages may not receive enough samples, breaking the connectivity of the roadmap.

The second part of the formal definition concerns any connected subset F_i of the free space. For PRM to successfully capture the connectivity of F_i , the volume of its β -lookout—points in F_i that can connect outside F_i —must be at least $\alpha \cdot \text{Vol}(F_i)$. Again, narrow passages reduce this lookout region, so even if samples exist within each large component, the PRM may not be able to link them due to insufficient samples at their boundaries.

Thus, even if the free space is connected, if ϵ , α , or β are small, PRM with uniform sampling may fail to generate enough useful samples to traverse critical narrow regions.

PART B

- 1) The configuration of a single drone is fully defined by its position $(x, y, z) \in \mathbb{R}^3$.
 - The **C-space** for centralised motion planning of K drones ($K \in [1, 12]$) is $C - \text{space} = \mathbb{R}^{3K}$ ($3K$ -dimensional C-space), where each configuration $Q = (x_1, y_1, z_1, \dots, x_K, y_K, z_K)$ represents the 3D positions of all drones in the team.
 - The **collision-free C-space** $C_{\text{free}} \subseteq C$ contains all configurations where
 - (1) each drone remains within the workspace boundaries;
 - (2) no drone collides with obstacles;
 - (3) no two drones collide with each other;
- 2) In my design, I implemented a centralised **bidirectional Rapidly-exploring Random Tree connect (RRT-connect)** planner in the C-space $\mathbb{R}^{3K}$. The **algorithm** simultaneously grows two trees: one from the initial configuration *start_config* and one from the goal configuration *goal_config*.

Pseudo-code:

INPUT:

$\text{start_config} \in \mathbb{R}^{3K}$	// Initial configuration of all drones $(x_1, y_1, z_1, \dots, x_K, y_K, z_K)$
$\text{goal_config} \in \mathbb{R}^{3K}$	// Goal configuration of all drones
$\text{max_iterations } N$	// Maximum number of planning iterations
$\text{step_size } \delta$	// Step size for steering toward sampled configurations
$\text{max_edge_length } D$	// Maximum allowed edge length
goal_threshold	// Goal threshold for reaching goal regions
$\text{connection_threshold}$	// Threshold for connecting two trees

OUTPUT:

A collision-free path from start_config to goal_config, or Failure

INITIALIZE:

```
T_start <- tree_with_root(start_config)
T_goal <- tree_with_root(goal_config)
iteration <- 0
```

MAIN LOOP:

While iteration < max_iterations:

```
// Uniformly sample a configuration in  $\mathbb{R}^{3K}$ 
q_rand <- SampleRandomConfiguration ()
```

```
// Extend the start tree toward the sampled point (detail in helper function below)
status, new_node <- EXTEND(T_start, q_rand)
```

```
if status ≠ "trapped": // trapped = extension fails
// Connect T_goal toward q_new using repeated extension
status_connect, goal_node <- CONNECT(T_goal, new_node)
```

```
// Check if the trees connect:
```

```
if status_connect == "reached" and CAN_CONNECT_TREES(new_node, goal_node):
// Extract the full path by merging both trees
path <- EXTRACT_BIDIRECTIONAL_PATH(new_node, goal_node)
```

```
// Check if the final configuration actually satisfies the goal condition
if VALIDATE_GOAL_REACH(path):
return path // Success
```

```
// Swap the roles of the trees for balanced growth
Swap(T_start, T_goal)
iteration <- iteration + 1
```

```
return Failure // No valid path found after max_iterations
```

Main helper function:

```
FUNCTION EXTEND(tree, q_rand):
```

```
q_near <- NearestNode(tree, q_rand) // Find a node nearest to q_rand
q_new <- Steer(q_near, q_rand, step_size) // Move toward q_rand by step_size
```

```
if Distance(q_near, q_new) > max_edge_length or not CollisionFree(q_near, q_new):
return "trapped", None // Can't extend due to edge length or collision
```

```
new_node <- AddNode(tree, q_new, parent = q_near) // Add new valid node
```

```
if IsNearGoal(q_new, goal_config):
return "reached", new_node // Close enough to target
```

```
else:
return "advanced", new_node // Partial progress made
```

3) Details on the code files.

4) **The primary objective** is to investigate how increasing environment complexity influences the performance of the centralized bidirectional RRT-Connect planner.

Experimental setup:

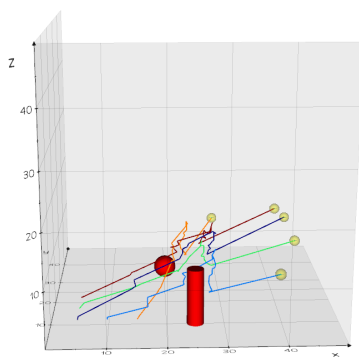
- **Controlled variable:** To ensure fair comparisons, the number of drones ($K = 5$) and their arrangement are fixed throughout all experiments.

- **Runs per setting:** 30 trials

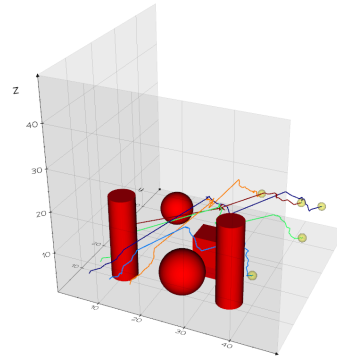
- **Independent environments:**

Four simulated environments were tested: (as shown below graph)

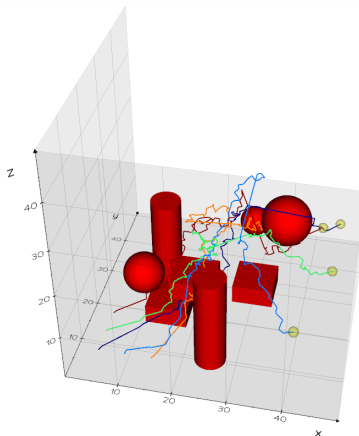
1. env_easy.yaml (easy environment with two simple obstacles)
2. env_medium.yaml (medium environment with 5 obstacles)
3. env_hard.yaml (hard environment with 8 obstacles)
4. env_narrow.yaml (narrow passage environment)



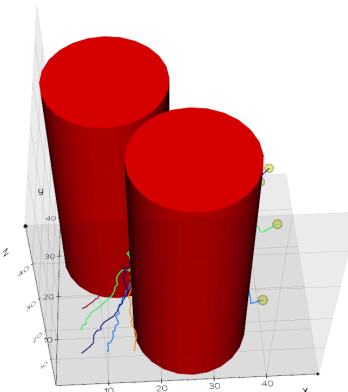
env_easy.yaml



env_medium.yaml



env_hard.yaml



env_narrow.yaml

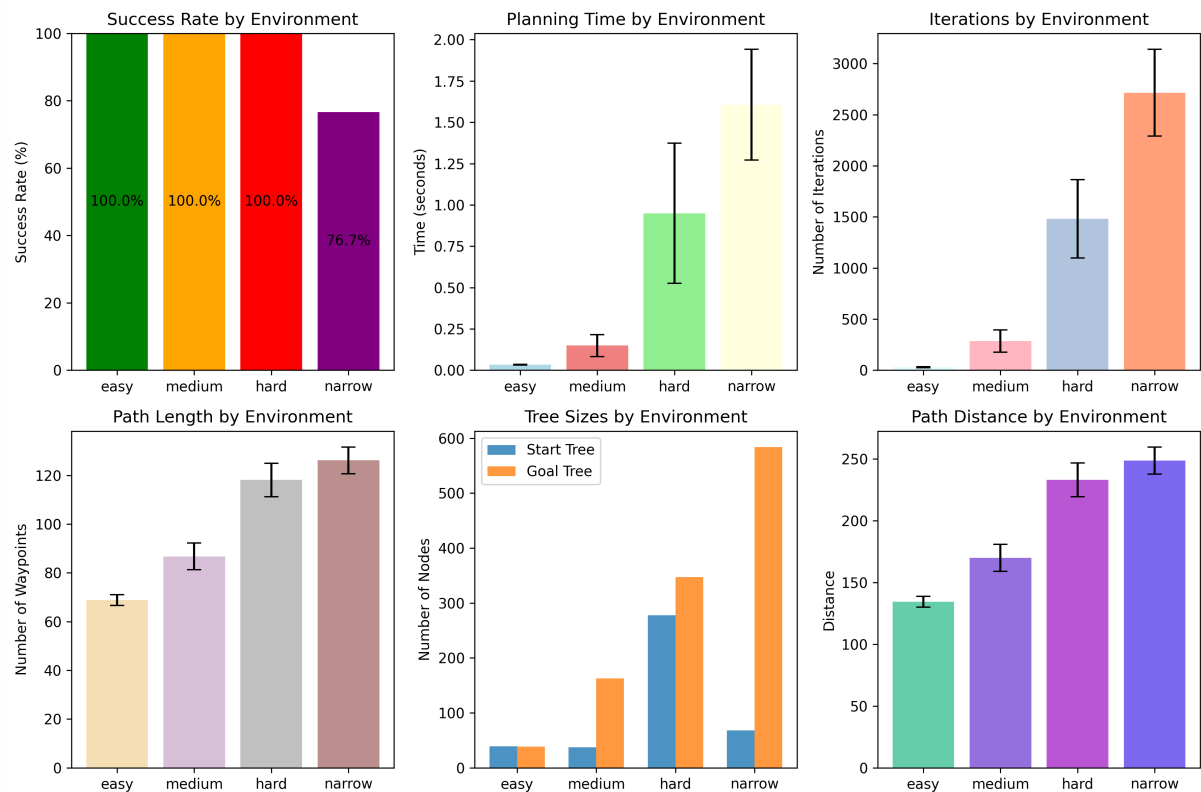
- **Metrics Recorded:**

1. Number of iterations;
2. Planning time (seconds);
3. Start tree size;
4. Goal tree size
5. Path length (number of waypoints);
6. Path distance (Euclidean length)

Summary statistics: (Successful runs only)

All values reported as: Mean $\pm$ 95% Confidence Interval

Environment	Success Rate	Iterations (Mean $\pm$ CI)	Planning Time (s)	Start Tree Size	Goal Tree Size	Path Length	Path Distance
Easy	100% (30/30)	29.27 $\pm$ 4.02	0.033 $\pm$ 0.002	39.23 $\pm$ 2.68	38.43 $\pm$ 2.02	68.87 $\pm$ 2.23	134.35 $\pm$ 4.39
Medium	100% (30/30)	284.93 $\pm$ 107.95	0.149 $\pm$ 0.066	37.40 $\pm$ 3.45	162.80 $\pm$ 54.84	86.77 $\pm$ 5.49	169.94 $\pm$ 10.97
Hard	100% (30/30)	1481.00 $\pm$ 384.28	0.949 $\pm$ 0.424	277.73 $\pm$ 132.56	347.23 $\pm$ 168.68	118.10 $\pm$ 6.90	233.16 $\pm$ 13.71
Narrow	76.7% (23/30)	2714.04 $\pm$ 424.20	1.606 $\pm$ 0.335	68.44 $\pm$ 4.86	583.91 $\pm$ 76.98	126.13 $\pm$ 5.47	248.77 $\pm$ 10.88



Analysis and Discussion

As environment complexity increases, the performance of the centralized bidirectional RRT-Connect motion planner declines in both efficiency and reliability. In simple environments like `env_easy`, the planner achieves a 100% success rate with minimal planning time and iterations, indicating that connections between start and goal configurations are easily established. However, as obstacles become more intricate and the layout grows more constrained (as seen in `env_medium` and especially `env_hard`) the planner requires significantly more iterations and planning time to find a valid path. Tree sizes also grow larger, reflecting the increased effort required to explore the configuration space and reach feasible connections.

The most dramatic decline occurs in the `env_narrow` environment, where the success rate drops to 76.7%. This drop highlights a known limitation of sampling-based planners: they struggle to sample valid configurations within narrow passages, which significantly reduces the chance of successfully connecting the two trees. Consequently, planning time and the number of iterations surge, and the goal tree grows disproportionately large as the

algorithm tries and often fails to connect through constrained regions. Additionally, as complexity rises, the resulting paths become longer and less direct, suggesting that the planner increasingly prioritizes feasibility over path optimality.

Overall, while the RRT-Connect planner performs robustly in open or moderately complex spaces, its performance deteriorates in environments with narrow corridors or dense obstacle configurations. This performance degradation is mainly due to the inherent challenges of uniform sampling in constrained areas and the increased difficulty in making successful tree connections.

- 5) **The primary objective** is to investigate how increasing the number of drones influences the performance of motion planner.

Experimental setup:

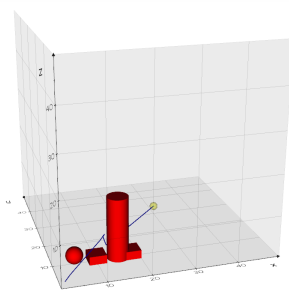
- **Runs per setting:** 30 trials

- **Controlled variable:**

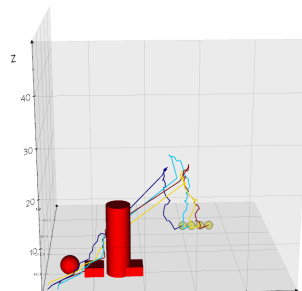
- 1) Fixed obstacle layout and environment.
- 2) Drone Arrangement: Drones start in a compact grid near the origin and aim for evenly spaced goals arranged in a straight line.

- **Independent variable:**

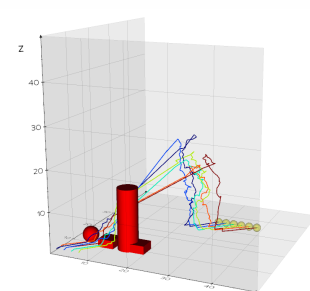
Number of drones: {1,4,6,8,10,12}.



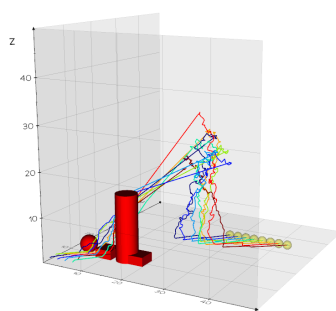
1 drone



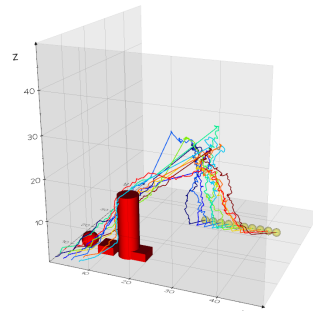
4 drones



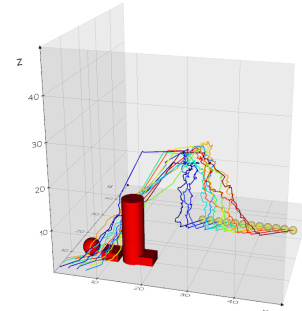
6 drones



8 drones



10 drones



12 drones

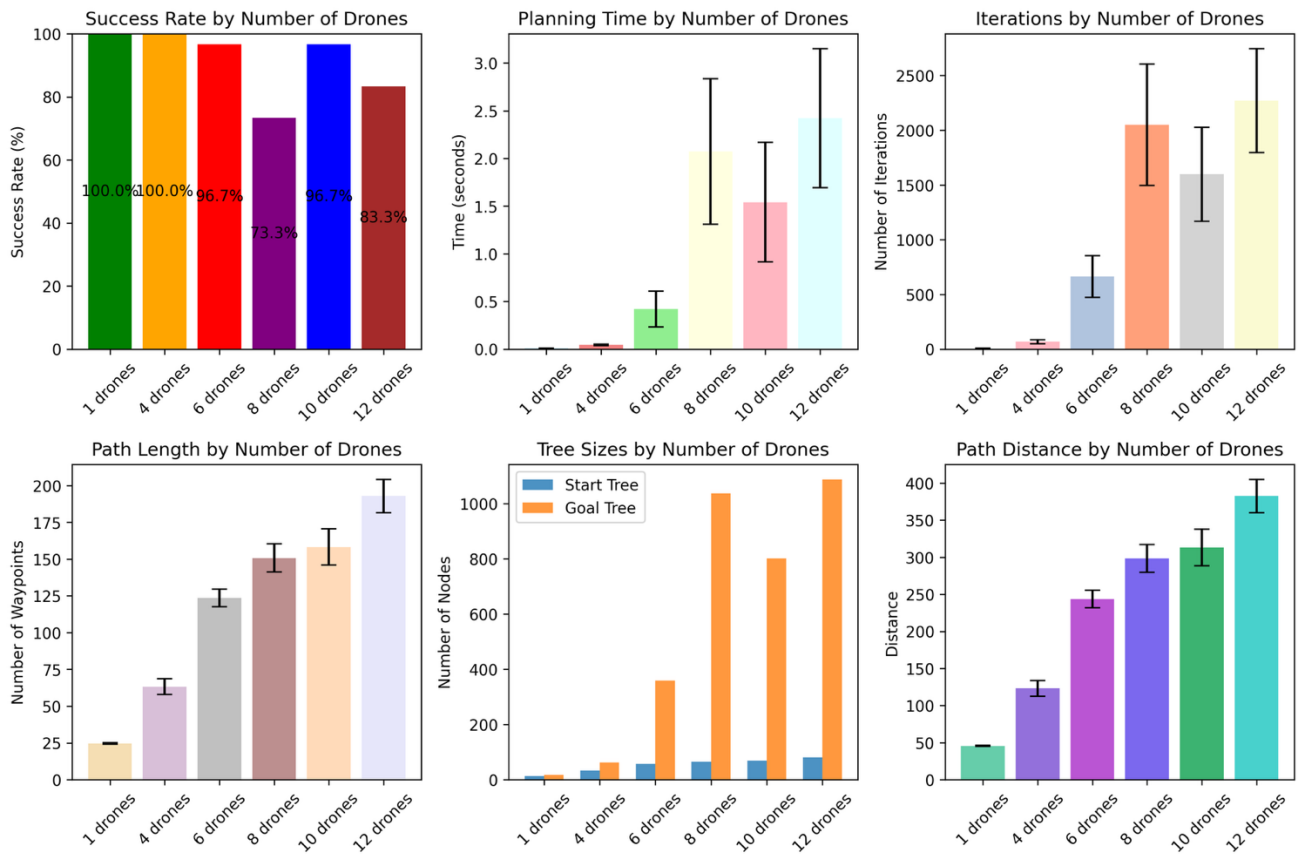
- **Metrics Recorded:**

1. Number of iterations;
2. Planning time (seconds);
3. Start tree size;
4. Goal tree size
5. Path length (number of waypoints);
6. Path distance (Euclidean length)

Summary Statistics: (Successful runs only)

All values reported as: Mean $\pm$ 95% Confidence Interval

# Drones	Success Rate	Iterations (Mean $\pm$ CI)	Planning Time (s)	Start Tree Size	Goal Tree Size	Path Length	Path Distance
1	100% (30/30)	6.27 $\pm$ 3.35	0.009 $\pm$ 0.001	12.80 $\pm$ 2.56	17.70 $\pm$ 1.82	24.60 $\pm$ 0.49	45.59 $\pm$ 0.85
4	100% (30/30)	68.47 $\pm$ 18.44	0.043 $\pm$ 0.007	32.90 $\pm$ 3.45	61.70 $\pm$ 8.72	63.20 $\pm$ 5.34	123.20 $\pm$ 10.62
6	96.7% (29/30)	663.35 $\pm$ 190.99	0.420 $\pm$ 0.187	57.35 $\pm$ 2.32	358.86 $\pm$ 95.73	123.66 $\pm$ 6.00	243.80 $\pm$ 11.91
8	73.3% (22/30)	2050.18 $\pm$ 553.24	2.074 $\pm$ 0.762	64.68 $\pm$ 3.34	1036.50 $\pm$ 271.30	150.77 $\pm$ 9.50	298.33 $\pm$ 18.92
10	96.7% (29/30)	1598.17 $\pm$ 429.79	1.542 $\pm$ 0.627	68.72 $\pm$ 5.46	800.69 $\pm$ 210.82	158.24 $\pm$ 12.29	313.10 $\pm$ 24.52
12	83.3% (25/30)	2269.64 $\pm$ 474.75	2.421 $\pm$ 0.729	80.36 $\pm$ 4.51	1087.52 $\pm$ 227.63	192.92 $\pm$ 11.30	382.53 $\pm$ 22.55



Analysis and Discussion

As the number of drones increases, the performance of the centralized bidirectional RRT-Connect motion planner deteriorates significantly across multiple metrics. While the planner achieves a perfect success rate for smaller teams of 1 and 4 drones, success rates drop notably beyond this point, with only 73.3% success for 8 drones and fluctuating success for 10 and 12 drones. This decline reflects the exponential growth in the configuration space dimensionality ($3K$ for K drones), which leads to greater computational complexity and more difficulty in finding collision-free paths.

Both the number of iterations and planning time grow rapidly with more drones, indicating that the planner needs substantially more effort to find valid solutions. The sizes of the search trees also increase sharply, particularly the goal tree, suggesting a growing imbalance in tree expansion and increasing difficulty in connecting the start and goal configurations. In addition, the path length and path distance also increased significantly, likely due to the need for more intricate coordination and avoidance of inter-drone collisions. These findings demonstrate that while the centralized RRT-Connect planner is effective for small drone groups, it faces scalability limitations as the number of drones increases, highlighting the challenges of centralized planning in high-dimensional, multi-agent systems.

- 6) The centralized bidirectional RRT-Connect method faces several important limitations. First, it suffers from scalability issues due to the curse of dimensionality: as the number of drones (K) increases, the configuration space grows to $3K$ dimensions, making random sampling and nearest-neighbour searches less effective. This leads to rapid growth in tree sizes, longer planning times, and reduced success rates for larger teams. Second, the method struggles with narrow passage problems, as coordinating multiple drones through tight spaces requires precise sampling that uniform random exploration rarely achieves. In addition, the planner is highly sensitive to parameters such as step size, connection thresholds, and sampling bias, which means a single setting may not work well across all environments or team sizes. Another limitation is the computational cost of collision checking, since each expansion requires verifying obstacle avoidance and pairwise separation among all drones, which scales quadratically with the number of agents. Finally, the method assumes drones are simple bounding spheres and ignores realistic dynamics and timing constraints, so some generated paths may not be directly executable in practice without additional post-processing.

Reference:

The use of AI:

- Gemini - <https://g.co/gemini/share/958afacab09b>
- ChatGPT - <https://chatgpt.com/share/68a6ad83-668c-800f-a4b0-15a7dc436ba4>